System Test Runbook · Cycle 0

Step-by-step testing for Saturday 17 October: install, keys, and 16 app test cases

AgriAgent Final Year Project · 16 October 2026

> **What this is.** This is **Amit's** work for this week (schedule tasks **T4.1** fresh-install smoke test and **T4.2** system test plan). No coding. You install the project exactly like a new user, connect the API keys, and run 16 test cases on the Streamlit app. For each one you compare what you **see** with the **expected** behaviour written here, and record the result.
>
> **Some tests will fail. That is the point.** A failed test is a bug found, and finding bugs is the whole job of a tester. Do not try to fix anything. Just record it precisely.

1 Plan for Saturday

Part	What	Time
A	Install the project by following the README word for word (T4.1).	1–1.5 h
B	Create your Groq and Cloudflare keys; run the provider check.	30 min
C	Run test cases TC-01 to TC-16 in the app, record each result.	2 h
D	File one GitHub issue per failure.	45 min
E	Post the summary in the group **by Saturday 8 pm**.	10 min
F	**Sunday:** write 10 test cases of your own (this becomes your T4.2 test plan).	1.5 h

Before you start: make a results sheet (Google Sheet or Excel) with these columns:

Test ID	Result	What actually happened (exact text / numbers)	Screenshot	Issue #
A1	Pass	Ran 175 tests in 3.4s, OK	A1.png	-
TC-xx	Fail	(example) Clicked Download: nothing happened; no file in Downloads; no message on screen	TCxx.png	#12

Name screenshots by test ID (`TC08.png`). The rows above are made-up examples of the format. Write **exact** numbers and messages, not "it worked".

2 Part A · Install (T4.1 smoke test)

Follow the README exactly. If a step fails, **write down the exact error**, take a screenshot, then use the fix given here so you can continue. Every step that needed a fix is a finding.

A1 · Get the code. New copy:

```
git clone https://github.com/Chaitanya-666/agriAgent.git
cd agriAgent
```

Already have it: `git checkout main` then `git pull`. **Expected:** latest commit is `fb31449` or newer (`git log --oneline -1`).

A2 · Python 3.11 environment.

```
# Linux / macOS
python3.11 -m venv .venv && source .venv/bin/activate
```

```
# Windows PowerShell
py -3.11 -m venv .venv ; .venv\Scripts\Activate.ps1
```

Expected: the prompt starts with `(.venv)`; `python --version` prints 3.11.x. Only 3.12 or newer installed? Record that as a finding and try with what you have. If PowerShell blocks the activate script, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.

A3 · Install (the README way).

```
python -m pip install -e ".[dev]"
```

Expected: finishes with `Successfully installed ...`. This downloads PyTorch and other large libraries (1–4 GB, 10–30 min). Record the time it took. **If it fails:** record the error, then install the light version that is enough for all of Part C:

```
pip install numpy pillow pyyaml pydantic chromadb streamlit requests python-dotenv opencv-
python
pip install -e . --no-deps
```

A4 · Settings file. `cp .env.example .env` (Windows: `copy .env.example .env`). **Expected:** a file `.env` now exists. Run `git status`: `.env` must **not** be listed (it is ignored). If it is listed, that is a high-severity finding.

A5 · Automated tests.

```
python -m unittest discover tests
python tests/e2e/runner.py --all
```

On Linux/macOS put `PYTHONPATH=.` in front of each; on Windows run `$env:PYTHONPATH="."` once first. **Expected:** first command ends with `Ran 175 tests ... OK`; second shows `Passed: 35`, `Failed: 0`, `Errors: 0`.

A6 · Start the app.

```
streamlit run app/app.py
```

Expected: the terminal prints a Local URL (`http://localhost:8501`) and the browser opens the dashboard within about 30 seconds. Keep this terminal visible all day: **its log is part of your evidence.**

3 Part B · Keys

Do Appendix A, sections A.2–A.4, of `docs/AgriAgent_Schedule.pdf` (Groq key, Cloudflare account, `.env`). Rules: **your own accounts, keys only in `.env`, never in WhatsApp or screenshots, never add a card.** Then in a **second** terminal (activate `.venv` again):

```
python scripts/check_providers.py groq
python scripts/check_providers.py cloudflare
```

B1 Expected: `[PASS] groq ... replied 'OK'`. **B2 Expected:** `[PASS] cloudflare ... clef-flash answers: {...}`. A FAIL with `HTTP 401` means the key was copied

wrongly; anything else, record it exactly. (The `local` and `clef-local` checks will FAIL on your laptop. That is expected; skip them.)

4 Part C · App test cases

Read once before starting.

- All cases run with **Mock Mode ticked** (sidebar, default) unless the case says otherwise. Mock mode shows fixed stand-in detections, not real AI output. That is fine: today we test the **app**, not accuracy.
- **Reset between cases:** put every sidebar control back to its default (Mock Mode ticked, Crop Cotton, prompt `weed . grass . pigweed .`, confidence 0.30, buffer 10 px, Force System 2 unticked) and remove any uploaded file, so "use sample field photo" is ticked again. Pressing F5 does this.
- The **four numbers** below the title are the KPI cards: Weed Infestation Area · Chemical Volume Saved · Target Weed Clusters · Spray Margin Buffer.

TC-01 App loads with correct defaults

Do: open `http://localhost:8501`. Look at the sidebar.
Expected (correct behaviour): title "AgriAgent: Dual-Brain …"; sidebar "Execution Controls" with **Mock Mode (Fast Laptop Demo)** ticked and an info box about mock mode; Crop **Cotton**; prompt `weed . grass . pigweed .`; confidence **0.30**; buffer **10 px**; Force System 2 unticked; "Or use sample field photo" ticked; no red error anywhere.
Record: Pass / Fail · what you actually saw · screenshot name

TC-02 Sample photo, default settings

Do: nothing; the sample runs automatically. Scroll through the page.
Expected (correct behaviour): left image: a green synthetic field; right image: same field with **3 boxes** labelled `weed` and coloured masks. KPI cards: **9.4%** · **87.8%** · **3** · **10 px**. (If you see **84.9%** instead of 87.8%, OpenCV is not installed: that is known issue 10. Record it, but do not file a new issue.)
Record: Pass / Fail · what you actually saw · screenshot name

TC-03 GeoJSON download contains only approved weed zones

Do: click **Download Field Prescription (GeoJSON)**. Open the file in a text editor (Notepad / VS Code).
Expected (correct behaviour): file name `agriagent_spray_prescription.geojson`; `"type": "FeatureCollection"`; **10** entries in `"features"`, each with `"target": "weed"`. **No herbicide name should appear**, because no prescription was requested (Force System 2 is off). Search the file for `active_ingredient` and record what you find.
Record: Pass / Fail · what you actually saw · screenshot name

TC-04 Crop label first in the prompt: nothing sprayed

Do: prompt `cotton . weed .` then press Enter.
Expected (correct behaviour): labels become `cotton`; KPI cards **0.0%** · **100.0%** · **0**. Download the GeoJSON: **0 features**. The file's `"status"` (inside `"metadata"`) must **not** claim a spray prescription was generated, since nothing will be sprayed. Record the exact status text.
Record: Pass / Fail · what you actually saw · screenshot name

TC-05 Label containing a crop word is never sprayed (fail-closed)

Do: prompt `cotton weed .`
Expected (correct behaviour): labels `cotton weed`; **0.0%** infestation, **0** clusters, GeoJSON with 0 features.
Record: Pass / Fail · what you actually saw · screenshot name

TC-06 Weed first, crop second

Do: prompt `weed . cotton .`
Expected (correct behaviour): same as TC-02: labels `weed`, **9.4%** · **87.8%** · **3**.
Record: Pass / Fail · what you actually saw · screenshot name

TC-07 Empty prompt

Do: delete all text from the prompt box, press Enter.
Expected (correct behaviour): the app should refuse or warn ("enter a detection prompt"). With no words there is nothing to detect, so it must **not** report weeds. Record what it does.
Record: Pass / Fail · what you actually saw · screenshot name

TC-08 Confidence slider affects the spray map

Do: move **Detection Confidence Threshold** to **0.90**.
Expected (correct behaviour): boxes below 0.90 disappear from the right image (the three boxes score 0.91, 0.86, 0.79, so one stays). **The KPI cards and the GeoJSON must change too**: 1 cluster and lower infestation. A spray map that still contains hidden boxes is a safety bug. Record all four KPI values and the GeoJSON feature count.
Record: Pass / Fail · what you actually saw · screenshot name

TC-09 Nozzle buffer changes savings, not infestation

Do: buffer **5 px**, then **30 px**.
Expected (correct behaviour): 5 px: **9.4%** · **89.2%**; 30 px: **9.4%** · **82.4%**. Bigger buffer = more area sprayed = less saved. Infestation stays 9.4%.
Record: Pass / Fail · what you actually saw · screenshot name

TC-10 Force System 2 for cotton

Do: tick **Force System 2 Statutory Consultation** (crop Cotton).
Expected (correct behaviour): a "System 2" section appears with a green status box; chemical **Pyrithiobac-sodium 10% EC**, dose **75 g a.i./ha**, a citation to the ICAR bulletin (page 35). Open "Visual Diagnostic Reasoning": the text starts with **[MOCK]**. Because the diagnosis is mock, the screen must make that clear to the user. Record the exact wording of the green status box and whether anything visible (not hidden in the expander) says "mock".
Record: Pass / Fail · what you actually saw · screenshot name

TC-11 Force System 2 for other crops

Do: Force System 2 ticked; set Crop to **Wheat**, then **Rice**, then **Sugar Beet**. Also download the GeoJSON for Wheat.
Expected (correct behaviour): a recommendation **for that crop**, or "no statutory evidence found, zero chemicals prescribed". It must **never** show a cotton herbicide or say "crop: cotton" when you selected Wheat. Check the GeoJSON too: `"crop"` and `"active_ingredient"` must agree. Record exactly what each crop shows.
Record: Pass / Fail · what you actually saw · screenshot name

TC-12 Real field photo in mock mode

Do: untick the sample box; upload `tests/fixtures/sample/cotton_20190613_6062W_CM_11.jpg` from the repo.
Expected (correct behaviour): green "image loaded" message; the photo shows with boxes. Because this is mock mode, the boxes are stand-ins: record whether they sit on actual weeds, and whether **the screen tells the user these are not real detections**. Repeat with `beet_X-10-0.png`; compare box positions with the cotton photo.
Record: Pass / Fail · what you actually saw · screenshot name

TC-13 Corrupt file is rejected cleanly

Do: make a text file, type anything, save it as `fake.jpg`; upload it.
Expected (correct behaviour): a red box "Image Decoding Error: Corrupt or invalid image payload …"; no crash, no traceback.
Record: Pass / Fail · what you actually saw · screenshot name

TC-14 Large phone photo

Do: upload a full-resolution photo from your phone (any plants).
Expected (correct behaviour): processes without crashing. Record the photo's size in MB, the time until results appear, and any warning.
Record: Pass / Fail · what you actually saw · screenshot name

TC-15 Live mode without model weights must not pretend

Do: untick Mock Mode; keep the sample photo. Watch the **terminal log** while it runs.
Expected (correct behaviour): one of two honest outcomes: (a) the real models download (about 1 GB on first use) and run, slowly on CPU; the log shows `Grounding DINO initialized successfully`; or (b) a clear error says the real models are unavailable. **Defect if:** the log shows `Falling back to mock_mode` while the screen shows ordinary results (9.4% · 87.8% · 3) **with no warning**. Copy the relevant log lines into your sheet.
Record: Pass / Fail · what you actually saw · screenshot name

TC-16 Run twice in a row

Do: with default settings, change the buffer to 11 then back to 10; then upload one photo, then a second photo.
Expected (correct behaviour): results update each time; second photo replaces the first; no stale image or numbers from the previous run.
Record: Pass / Fail · what you actually saw · screenshot name

5 Part D · Filing defects (GitHub Issues)

One issue per failed test case at `https://github.com/Chaitanya-666/agriAgent/issues/new`. Search first; do not file duplicates. Use this exact shape:

```
Title:  [TC-xx] Download button does nothing   (made-up example)

Steps:     1. Start app, Mock Mode on, sample photo  2. Click "Download Field Prescription"
Expected:  file agriagent_spray_prescription.geojson is saved
Actual:    nothing happens; no file, no message
Evidence:  screenshot attached (TCxx.png), terminal log lines pasted below
Env:       Windows 11, Python 3.11.9, commit fb31449, Mock Mode on
Severity:  medium (export broken, workaround: none known)
```

Severity guide: high = something could be sprayed that should not be, mock output could pass as real, or the app crashes in the demo flow · **medium** = wrong or misleading number or text, with a workaround · **low** = cosmetic. If you cannot add labels, put the severity in the title. Chaitanya will assign each issue to its owner; when a fix is merged, **you** re-test it and close it.

6 Part E · Report in the group (Saturday by 8 pm)

```
T4.1 / Cycle 0 done.
Install: A1-A6 -> 5 pass, 1 fail (A3: <short reason>), install took 22 min
Keys: groq PASS, cloudflare PASS
App: 16 cases -> 11 pass, 5 fail
Issues filed: #5 #6 #7 #8 #9
Results sheet: <link>
```

7 Part F · Sunday: your own 10 test cases (T4.2)

Your system test plan = TC-01 to TC-16 above + **10 cases you design yourself**. Write each in the same form (ID, do, expected). Ideas to start from; pick and expand, do not copy:

- Each crop in the dropdown **without** Force System 2: what does the GeoJSON say?
- A PNG with a transparent background; a black-and-white photo; a tiny 50 × 50 px image.
- A prompt of 30 words; a prompt with Marathi or Hindi words; `weed .` alone.
- Buffer at the extremes (5 and 30) together with Force System 2.
- Download the GeoJSON twice: are both files identical?
- Two browser tabs open at once with different settings.
- Kill the terminal while a run is in progress, restart: does the app recover?
- With keys removed from `.env`, tick Force System 2: what does the user see?

Send the 26-case plan to Chaitanya by **Sunday 6 pm** for review. Next week the same cases are run again with the **real** models (Cycle 1), so write them clearly enough that Sahil could run them without asking you.